

Schedule Stabilization Report

Schedule Crash Stabilization and Schedule Accuracy & Adherence

30 Sep 2026

Date

jchris_time-table-app

Branch (base 4b7d3dd)

Bus B · ThingsBoard Cloud

Test data (real payload)

9

Crash & state fixes,
including 2 ANRs and
MQTT

9

Schedule accuracy fixes

29/29

Unit tests passing

27/27

Trip/REP launches get the
correct route (real
payload)

AREA	STATUS
Crashes and invalid state in the schedule flow	Done tested on the emulator with real Bus B data
Single source for Ahead / On Time / Behind status	Done text, icon, color, zone, and log from one result
Next run / "1200+ mins"	Done
ETA and timing points explainable from the log	Done distance along the route
ANR (app freeze) caused by MQTT	New finding fixed
Live test with real GPS on a physical tablet	Not yet

Evidence note: screenshots were taken on an **Android tablet emulator** (Android 35, 2560×1600) running a debug build from this repo, connected to ThingsBoard Cloud as Bus B. No physical tablet was connected to the PC during this session, so evidence from a physical tablet still needs to be captured during the live test.

01 Crash & state fixes

C1 Completed trips reappear

BEFORE

Admin polling every 5 seconds overwrote progress with the full roster.

AFTER

The server roster is accepted once per session. Fetch Roster = a fresh snapshot. During a shift, local progress is not overwritten by polling (design decision).

ScheduleActivity.kt · ScheduleViewModel.kt

C2 Items lost on cancel / crash

BEFORE

The item was removed and the cache written before the driver committed.

AFTER

Removal is only staged. The cache is written on `RESULT_OK`. On cancel, the list is restored.

ScheduleActivity.kt

C3 Trip completed, process died, trip repeated

BEFORE

If the process died before `RESULT_OK` arrived, the trip came back.

AFTER

Map, REP, Break, and Signing write the cache themselves the moment the item completes (`ScheduleCache`).

ScheduleCache.kt (new) · Map/Rep/Break/Signing Activity

C4 REP given the Sign On placeholder route

BEFORE

The depot→depot REP and the Sign On placeholder share the same terminals, and the matcher took the first one.

AFTER

The route at the roster item's position is preferred, including for a trimmed roster (cache mode).

RouteMatcher.kt (new) · ScheduleActivity.kt

C5 Silently used the first route

BEFORE

`firstOrNull()` fallback, so the driver could follow another trip's stops.

AFTER

Launch is blocked, a toast is shown, and it is logged. Never falls back.

ScheduleActivity.kt

C6 Crashes on empty / malformed data

BEFORE

`first()`, `!!`, `toInt()` without guards.

AFTER

`firstOrNull()`, `toIntOrNull()`, `?: return`.

Map/Rep Activity · ViewModel · StatusManager

C7 ANR: MQTT subscribe new

BEFORE

"Connection lost" was thrown on the main thread. The main thread died and the UI froze.

AFTER

Subscribe is guarded. `App.kt` no longer swallows errors on the main thread.

MqttManager.kt · App.kt

C8 ANR: publish blocks the UI new

BEFORE

Synchronous `MqttClient.publish` from the main thread, waiting more than 5 seconds.

AFTER

Publish/subscribe run on a background thread, `timeToWait` = 3 seconds.

MqttManager.kt

C9 MQTT "rate limit": sessions kicking each other off new

BEFORE

ThingsBoard allows only one MQTT session per device token. ClientAttributesService and ScheduleActivity (then Map/REP) opened clients with the same Bus token at the same time, so they disconnected each other every 1–2 seconds. The client's fix in `c0230d1` (leaked publish loops) was correct, but not sufficient: still 225 disconnects in 6 minutes.

AFTER

The service only connects when it needs to publish, then disconnects. Schedule releases its connection while a trip screen is open and reconnects on return. A client that is mid auto-reconnect is now really stopped. Result: **0 disconnects** in the same flow.

BUILD	MQTT DISCONNECTS (~6 MINUTES)	PER MINUTE
Client branch <code>c0230d1</code>	225	38.6
+ schedule fixes	224	38.3
+ MQTT session fix (C9)	0	0

ClientAttributesService.kt · ScheduleActivity.kt · MqttManager.kt

02 Schedule accuracy fixes

Single status source: centralized thresholds

All thresholds live in `ScheduleAdherence.kt`. Text, icon, color, marker/zone, and log all use one `calculateScheduleStatus()` result, in both Map and REP. Delta = scheduled – predicted arrival.

Early delta > +30 s	On Time –30 ... +30 s	Slightly Behind –300 ... –31 s	Very Behind < –300 s
-------------------------------	---------------------------------	--	--------------------------------

A 10-second hysteresis at each boundary prevents the status from flickering due to GPS noise.

#	PROBLEM BEFORE	FIX
A1	Duplicate thresholds: internal <code>>0 / –300 / –600</code> , UI <code>±30 / –300</code> . +15 seconds = EARLY internally but "On Time" in the UI.	A single <code>ScheduleAdherence</code> module, used by Map and REP.
A2	Status flickered due to GPS noise	10-second hysteresis
A3	Next run used <code>list[1]</code> and could point to the wrong run. The "last run" dialog appeared while a run was still left.	<code>nextRunAfterActive()</code> selects the next run explicitly
A4	"Next run in 1250 mins", "Late by 1200+ mins"	No longer shifted to tomorrow. A difference of more than 4 hours is treated as stale data and shown as "Next run at HH:mm".
A5	Late formula jumped: +10 s became "late 10s", –10 s became "late 310s"	Shortfall against the minimum layover (300 s, value from the old code)
A6	Multiple clocks mixed, midnight not handled	One clock for all calculations. Time parsing is safe across midnight, malformed input is skipped.
A7	ETA used straight-line distance. On winding routes the distance actually increased as the bus got closer.	Remaining distance measured along the route polyline
A8	Fallback speed = distance to the timing point divided by the time of one full trip	Divided by the scheduled time to that same timing point. <code>speedSource</code> is logged.
A9	Logs were not enough to explain the status	One <code>ScheduleAdherence</code> line with all inputs and the result, no tokens

03 Validation

Unit tests (JDK 17)

SUITE	RESULT	COVERAGE
ScheduleAdherenceTest	18/18	§18A cases (Ahead +120, On Time –10, Behind –180), threshold boundaries, hysteresis, midnight, next run, countdown, speed source
RouteMatcherTest	5/5	REP vs Sign On placeholder, trimmed roster, real Bus A + B payload: 27 correct launches
MapActivityTest (existing)	5/5	No regressions
ExampleUnitTest	1/1	

State scenarios on the emulator (Bus B, real ThingsBoard)

SCENARIO	RESULT
Fetch Roster Bus B	Pass 20 items shown
Sign On completed, then wait 15 seconds of polling	Pass list starts from REP, not reset
Force-stop, then Use Cache	Pass Sign On does not repeat
Process killed in the middle of REP	Pass REP still on top, app automatically enters Schedule
Process killed while the "Trip Completed" dialog is showing	Pass trip does not repeat (twice: REP and 50B)
Route for REP 06:29	Pass selectedIdx=1 (previously 0)
90-second simulation after the MQTT fix	Pass 0 ANRs
Cancel on the Confirmation screen	Not testable a single checkbox launches immediately

Example: status traceable from the numbers in the log

```
scheduled=07:34:00 now=07:31:19 distanceRemainingM=1437 effectiveSpeedKmh=20.0
speedSource=GPS predictedArrival=07:35:38 deltaSec=-98 status=SLIGHTLY_BEHIND
```

1437 m ÷ 5.56 m/s = 259 s → 07:31:19 + 4:19 = 07:35:38 → 07:34:00 – 07:35:38 = –98 s →

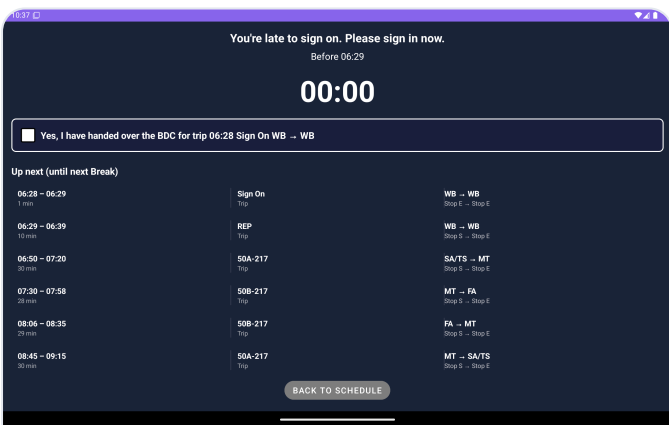
Slightly Behind

During the live test: `adb logcat -s ScheduleAdherence` , then match the status in the UI against the numbers in the log.

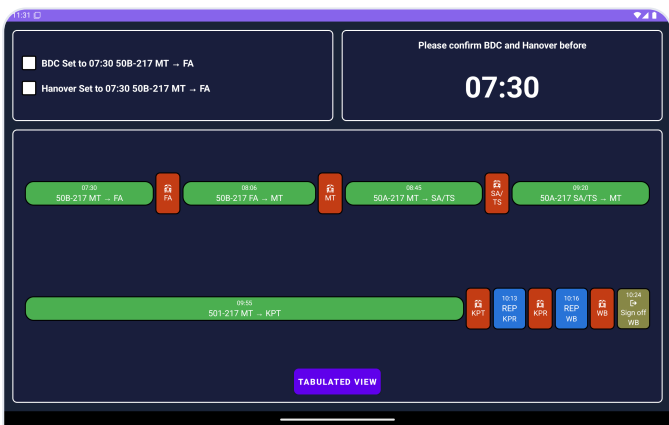
04 Evidence: schedule flow



Fetch Roster: the tablet is recognized as Bus B and 20 schedule items from ThingsBoard are shown.



Signing: Sign On. After Done, the cache is committed and the list moves on to REP.



Confirmation: 50B 07:30, route matched via RouteMatcher.

11:31

2026-09-30 11:31:18

TODAY'S OVERVIEW

COPY AID Online

07:30
50B-217 MT → FA

FA

08:06
50B-217 FA → MT

MT

08:45
50A-217 MT → SA/TS

SA/TS

09:20
50A-217 SA/TS → MT

MT

09:55
501-217 MT ...

KPT

10:13
REP
KPR

10:16
REP
KPR

WB

10:24
Sign off
WB

WB

10:29
WB

WB

11:15
Sign on
WB

WB

11:16
REP
WB

WB

11:21
REP
KPR

11:30
501-217 KPT → MT

MT

11:55
502-217 MT → MR

MR

12:33
REP
MR

MR

12:34
REP
WB

WB

12:43
Sign off
WB

Light Mode

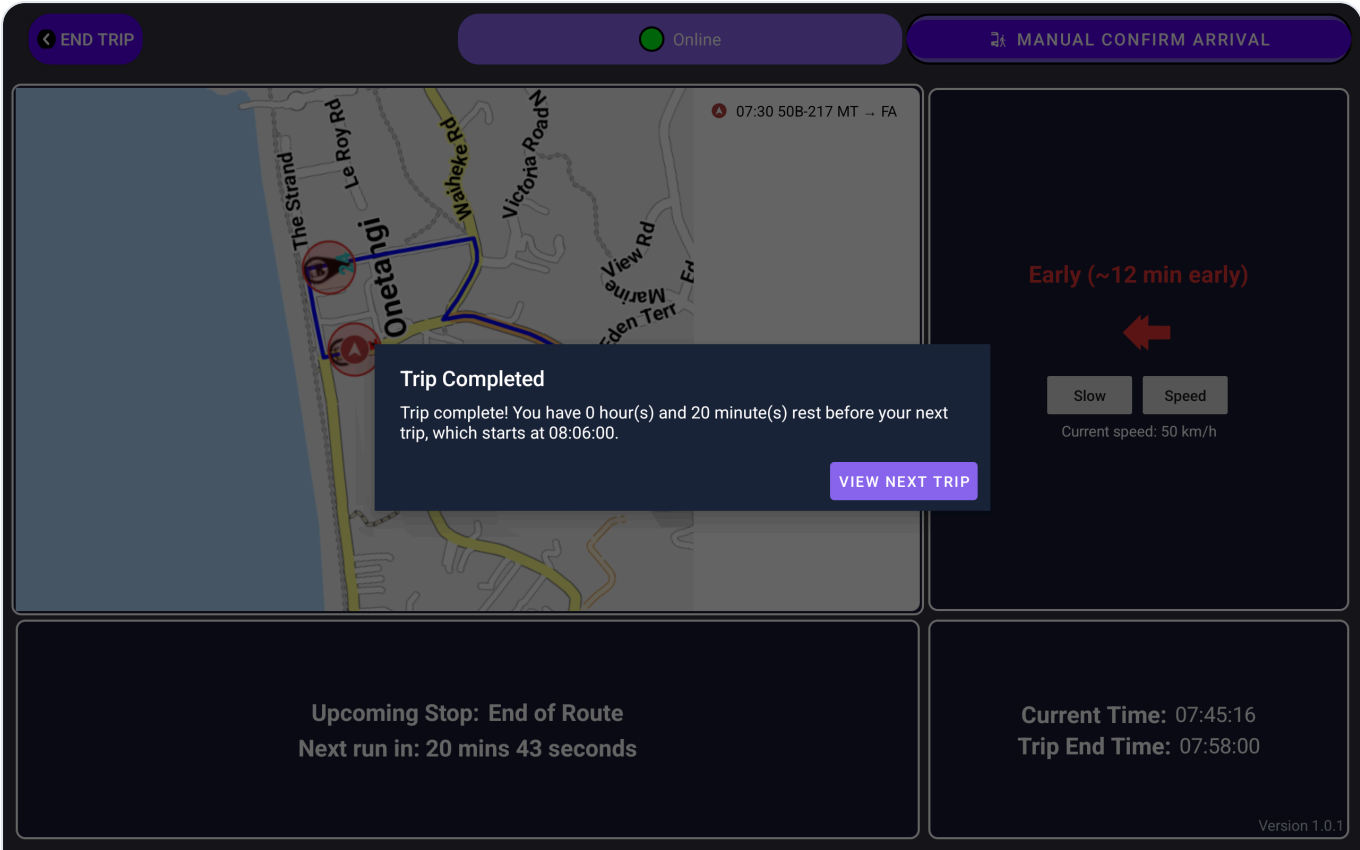
START ROUTE

TEST ROUTE

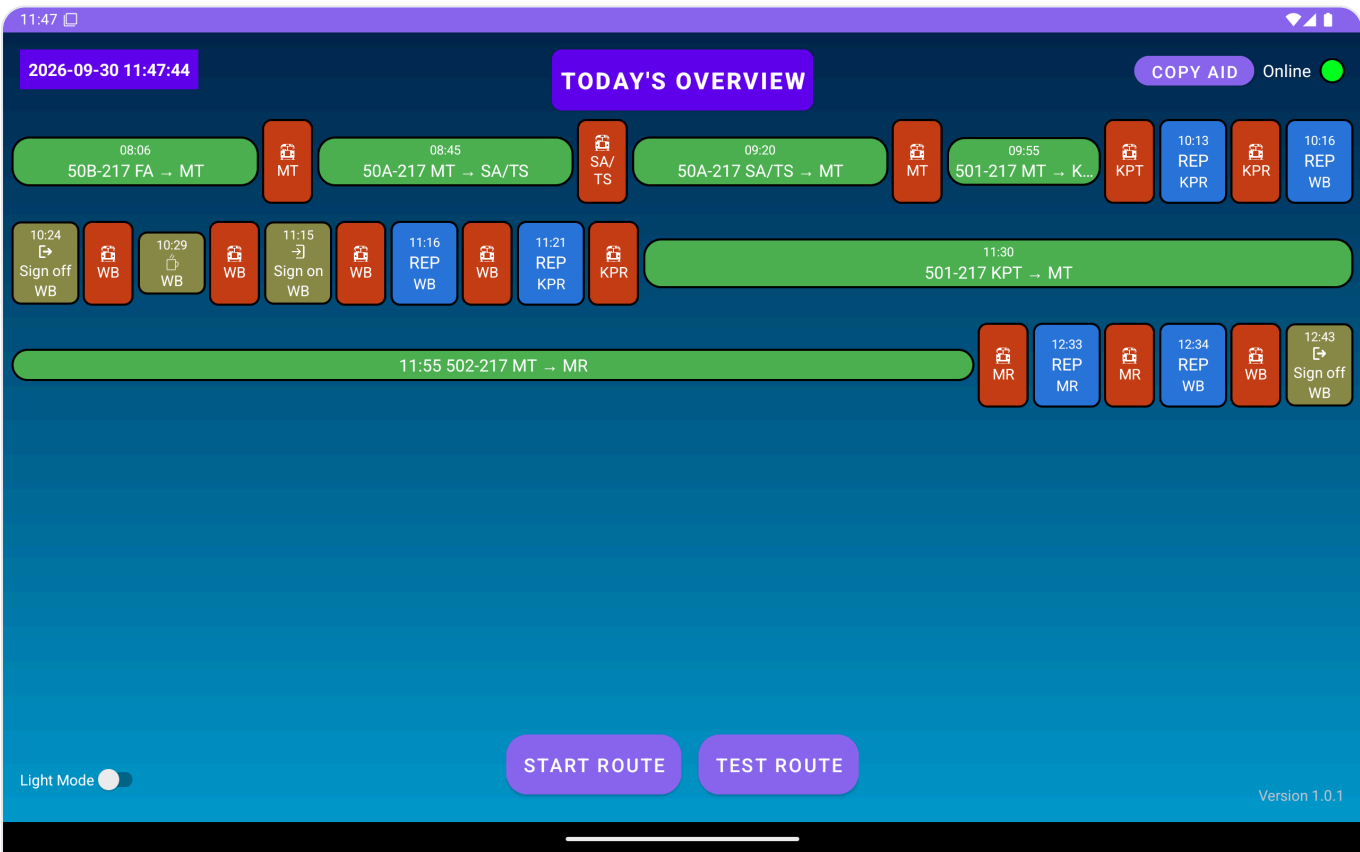
Version 1.0.1

After 50A completes: back to Schedule, 50B 07:30 first in the list, and the old trip does not reappear.

05 Evidence: completed, then process killed



Trip Completed: next trip 08:06 and a 20-minute break (correct next run). Log: MapActivity: Committed schedule cache after completion | remaining=16



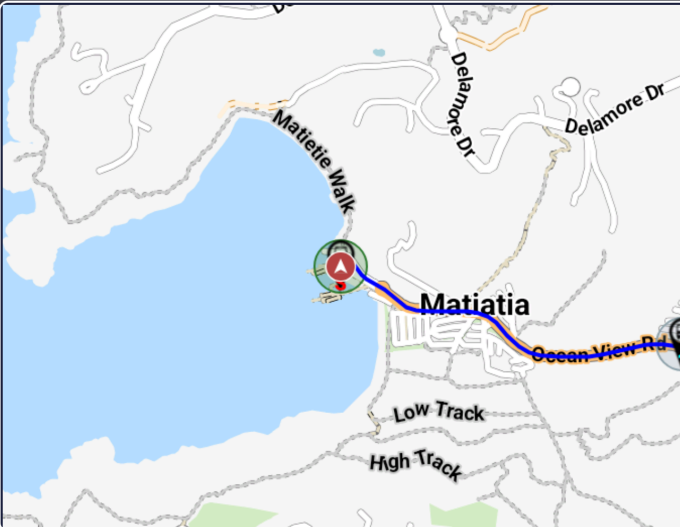
Process killed during the dialog, then reopened: list starts from 50B 08:06. The 07:30 trip **does not repeat**.

06 Evidence: adherence status

END TRIP

Online

MANUAL CONFIRM ARRIVAL



07:30 50B-217 MT - FA

On Time

Slow

Speed

Current speed: 0 km/h

Upcoming Stop: Owhanake Car Park
Next run in: 35 mins 51 seconds

Current Time: 07:30:08
Trip End Time: 07:58:00

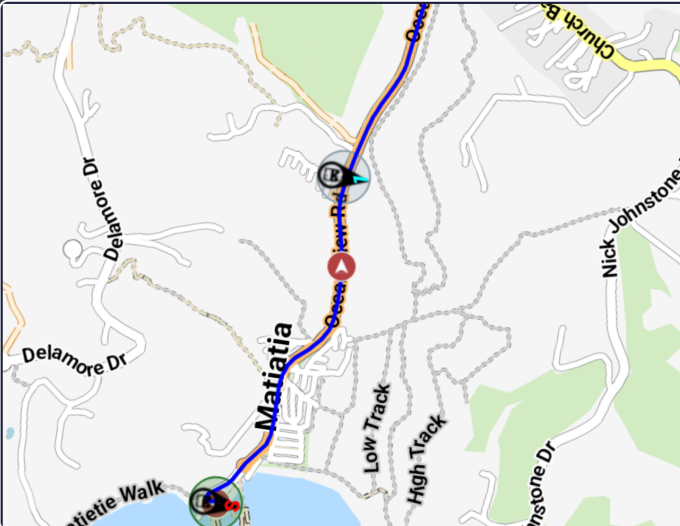
Version 1.0.1

Start of trip, speed 0: On Time, speedSource=SCHEDULE_AVERAGE_FALLBACK , deltaSec=-9 .

END TRIP

Online

MANUAL CONFIRM ARRIVAL



07:30 50B-217 MT - FA

Slightly Behind (~1 min late)

Slow

Speed

Current speed: 20 km/h

Upcoming Stop: Owhanake Car Park
Next run in: 34 mins 40 seconds

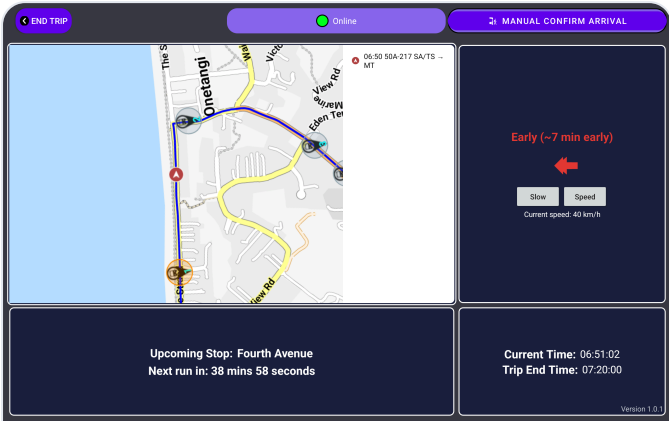
Current Time: 07:31:19
Trip End Time: 07:58:00

Version 1.0.1

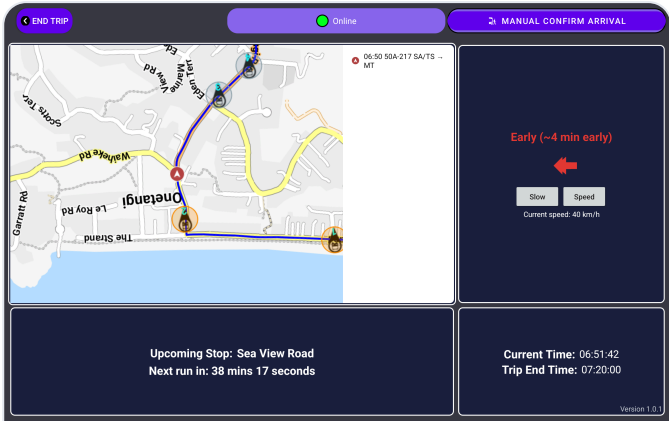
20 km/h: "Slightly Behind (~1 min late)" in blue, matching the log deltaSec=-98 status=SLIGHTLY_BEHIND .

07 Evidence: before vs after

ETA: straight line vs along the route

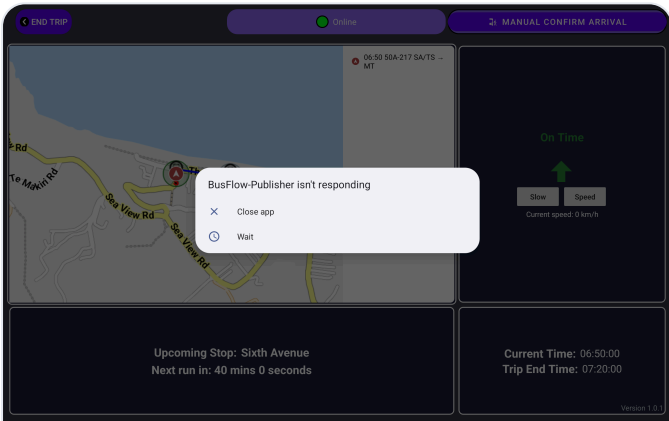


Before: straight-line distance, "Early ~7 min". The distance increased even though the bus was getting closer.

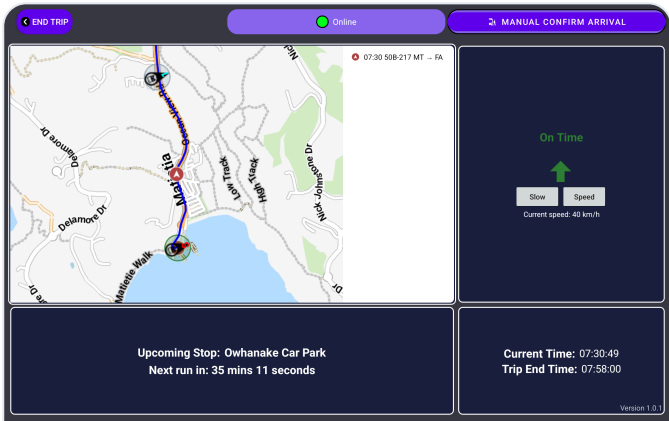


After: along the route, "Early ~4 min", and the prediction is stable at constant speed.

ANR caused by MQTT



Before: "BusFlow-Publisher isn't responding" when the MQTT connection drops.



After: the app stays responsive. Delta +39 s stays On Time due to hysteresis (needs more than 40).

08 Needs client confirmation

1. **Meaning of the zone colors.** Blue = Slightly Behind (30 seconds to 5 minutes). Previously the marker/zone used blue for 5 minutes or more late. Do the colors carry a specific operational meaning?
2. **5-minute minimum layover** (`NEXT_RUN_MIN_LAYOVER_SECONDS`). The value comes from the old code. Is this an operational rule?
3. **On Time tolerance of ± 30 seconds.** If the client uses, for example, ± 120 seconds, it only needs changing in one place.
4. **Schedule updates mid-shift** require another Fetch Roster.

09 Remaining risks & deferred items

MQTT disconnect/reconnect loop: fixed (C9). The cause was several clients with the same device token disconnecting each other's sessions. After the fix: 0 disconnects on the emulator. Still needs confirming on a physical tablet.

Remaining risks

- No live test with real GPS on a physical tablet yet.
- If the roster changes, the real-payload test needs to be rerun.
- Cancelling via Confirmation is not tested end to end.
- If the GPS jumps far, the distance estimate can be briefly wrong.
- Midnight and Very Behind are only covered by unit tests so far.

Deferred (out of scope)

- Config Data token is hardcoded in `MqttManager.kt`.
- `vlrskey.jks` is committed to the repo.
- Splash can hang because of the screen recording permission.
- `RepActivity` replaces the global crash handler.
- Duplicated code between Map and REP.

10 Next steps

1. Client confirmation for the points in section 08.
2. Commit to the `fix/schedule-stabilization` branch, one commit per logical change.
3. Install the APK on a physical tablet, get the AID, then set the Bus B AID in the `config` of all three devices (requires approval).
4. Live test with real GPS while recording the `ScheduleAdherence` log, then match the UI against the numbers in the log.

Full technical details (files per change and a chronology of the work) are in `docs/SCHEDULE_STABILIZATION_REPORT.md`.